Lista
Técnicas de Programação para Sistemas Embarcados I
Prof. Thiago Werlley
1º Semestre de 2025

Nome: Ana Alicy Ribeiro Dos Santos
Curso: Engenharia de Computação

1. Crie uma macro chamada CLEAR_BIT(x, b) que limpe o bit na posição b da variável x.

```
#define CLEAR_BIT(x, b) ((x) &= ~(1 << (b)))
```

Explicação:

- 1 << b cria um número com 1 no bit b.
- ~ inverte esse valor, deixando todos os bits como 1, exceto o b.
- O &= aplica o AND bit a bit, limpando o bit b.

exemplo:

```
#include <stdio.h>

#define CLEAR_BIT(x, b) ((x) &= ~(1 << (b)))

int main() {
   unsigned char x = 0b11111111;
   CLEAR_BIT(x, 3); // zera o bit 3
   printf("Resultado: 0x%02X\n", x); // Esperado: 0xF7
   return 0;
}
```

2. Crie uma macro chamada swap_bits(x, i, j) que troque os bits nas posições i e j da variável x.

```
#define swap_bits(x, i, j) \
   do { \
      if (((x >> i) & 1) != ((x >> j) & 1)) { \
         x ^= (1 << i) | (1 << j); \
      } \
   } while (0)
```

Explicação:

- Se os bits nas posições i e j forem diferentes, invertemos ambos usando XOR.

Exemplo:

```
#include <stdio.h>

#define swap_bits(x, i, j) \
   do { \
      if (((x >> i) & 1) != ((x >> j) & 1)) { \
         x ^= (1 << i) | (1 << j); \
      } \
   } while (0)

int main() {
   unsigned char x = 0b10000001;
   swap_bits(x, 0, 7); // troca os extremos
   printf("Resultado: 0x%02X\n", x); // Esperado: 0x81
   return 0;
}
```

3. Considere unsigned char status = 0b11101111;. Escreva uma linha de código que zere o bit 4.

```
status &= ~(1 << 4);
```

Explicação:

- 1 << 4 = 0b00010000
- ~(1 << 4) = 0b11101111
- status &= 0b11101111 → zera o bit 4

Exemplo:

```
#include <stdio.h>

int main() {
   unsigned char status = 0b11101111;
   status &= ~(1 << 4); // zera o bit 4
    printf("Resultado: 0x%02X\n", status); // Esperado: 0xEF (sem mudança visível pois já estava 0)
   return 0;
}
```

4. Escreva uma função unsigned char mirror_bits(unsigned char value) que inverta os bits de um byte.

```
unsigned char mirror_bits(unsigned char value) {
```

```
    unsigned char result = 0;
    for (int i = 0; i < 8; ++i) {
        result |= ((value >> i) & 1) << (7 - i);
    }
    return result;
}
```

Explicação:

- Lê os bits de value da direita e escreve invertido em result.

Exemplo:

```
#include <stdio.h>

unsigned char mirror_bits(unsigned char value) {
    unsigned char result = 0;
    for (int i = 0; i < 8; ++i)
        result |= ((value >> i) & 1) << (7 - i);
    return result;
}

int main() {
    unsigned char original = 0b00010110; // espelho esperado: 01101000
    unsigned char mirrored = mirror_bits(original);
    printf("Original:  0x%02X\n", original);
    printf("Espelhado: 0x%02X\n", mirrored);
    return 0;
}
```

Saída:

```
Original:  0x16
Espelhado: 0x68
```

7. Implemente uma função unsigned int circular_right(unsigned int x, int n) que rotacione x para a direita n vezes.

```
unsigned int circular_right(unsigned int x, int n) {
    n %= 32;
    return (x >> n) | (x << (32 - n));
}
```

Explicação:

- Roda x para a direita n vezes, reinserindo os bits deslocados à esquerda.

Exemplo:

```
#include <stdio.h>

unsigned int circular_right(unsigned int x, int n) {
```

```
    n %= 32;
    return (x >> n) | (x << (32 - n));
}

int main() {
    unsigned int val = 0x80000001; // 10000000...00000001
    unsigned int rot = circular_right(val, 1);
    printf("Original: 0x%08X\n", val);
    printf("Rotacionado: 0x%08X\n", rot); // Esperado: 0xC0000000
    return 0;
}
```

Saída:
Original: 0x80000001
Rotacionado: 0xC0000000

12. Qual a saída gerada pelo Código 1? Justifique sua resposta.

```c
#include <stdio.h>
#define FIRST_PART  7
#define LAST_PART   5
#define ALL_PARTS FIRST_PART + LAST_PART
int main() {
  printf("The square of all the parts is %d\n", ALL_PARTS * ALL_PARTS);
  return (0);
}
```

Código 1

Marque a resposta correta:
(a). 12
(b). 24
(c). 47
(d). 144

O que o pré-processador vê:

printf("The square of all the parts is %d\n", 7 + 5 * 7 + 5);

Avaliando:
Multiplicação tem precedência:

7 + (5 * 7) + 5 = 7 + 35 + 5 = 47

Alternativa correta: (c) 47

18. O Código 7 calcula o valor errado para SIZE. Porque?

```c
#include <stdio.h>
#define SIZE 10;
#define FUDGE SIZE -2;

int main(){
  int size;

  size = FUDGE;
  printf("Size is %d\n", size);
  return (0);
}
```

Código 7

Erro: O ponto e vírgula está sendo incluído na substituição do pré-processador!

Resultado expandido:

```
int size;
size = 10; -2;  // Erro de sintaxe!
```

Correção:

```
#define SIZE 10
#define FUDGE (SIZE - 2)
```

Explicação:

- Macros não devem terminar com ; — elas são apenas substituições de texto.

Exemplo com correção:

```
#include <stdio.h>

#define SIZE 10   // sem o ponto e vírgula!
#define FUDGE (SIZE - 2)

int main() {
   int size;
   size = FUDGE;
   printf("Size is %d\n", size); // Esperado: 8
   return 0;
}
```

20. Encontrar o bitwise e, ou e xor dos seguintes casos:
(a) 0xC6 com 0x35

Binário:
0xC6 = 11000110
0x35 = 00110101

Cálculos:
AND(&): 11000110

Regra:

- O resultado é 1 somente se os dois bits forem 1.
- Caso contrário, é 0.

```
  11000110   (0xC6)
& 00110101   (0x35)
------------
  00000100   → 0x04
```

Só o bit 2 (contando da direita, a posição 2) é 1 nos dois números.

Bitwise OR (|)
Regra:

- O resultado é 1 se pelo menos um dos bits for 1.

```
  11000110
| 00110101
------------
  11110111   → 0xF7
```

Todos os bits que são 1 em qualquer um dos dois números viram 1 no resultado.

Bitwise XOR (^)
Regra:
O resultado é 1 se os bits forem diferentes (um for 0 e o outro 1).

```
  11000110
^ 00110101
------------
  11110011   → 0xF3
```

Os bits onde os números são diferentes viram 1, onde são iguais (0 e 0 ou 1 e 1), viram 0.

<u>Exemplo:</u>

```
#include <stdio.h>

int main() {
   unsigned char a = 0xC6;
   unsigned char b = 0x35;

   printf("AND: 0x%02X\n", a & b); // 0x04
   printf("OR : 0x%02X\n", a | b); // 0xF7
   printf("XOR: 0x%02X\n", a ^ b); // 0xF3
   return 0;
}
```

<u>Saída:</u>

```
AND: 0x04
```

OR : 0xF7
XOR: 0xF3

25. Escreva um programa que pegue nos bits de um número e os desloque para a extremidade esquerda. Por exemplo, 01010110 (binário) tornará 11110000 (binário).

Exemplo:

```
#include <stdio.h>

unsigned char shift_bits_left(unsigned char value) {
    int count = 0;
    for (int i = 0; i < 8; ++i)
        if ((value >> i) & 1)
            count++;
    return 0xFF << (8 - count);
}

int main() {
    unsigned char value = 0b01010110; // tem 4 bits 1
    unsigned char result = shift_bits_left(value);
    printf("Resultado: 0x%02X\n", result); // Esperado: 0xF0
    return 0;
}
```

Saída:

```
Resultado: 0xF0
```